

SURICON 2026 · HANDS-ON LAB

When Suricata Breaks

Hands-On Detection Engineering and IDS Operations Recovery

Ted Skinner

Security Architect, Cisco

About your speaker



Ted Skinner

Security Architect, Cisco

- Security monitoring, intrusion detection, AI SOC
- Detection engineering
- Works with Suricata, Splunk Enterprise Security, SOAR, Zeek and Sysmon

Workshop materials
github.com/tskinnerarlo/When-Suricata-Breaks

What we will cover

1	**Why sensors fail quietly**	The problem, and why "running" is not "healthy"	6 min
2	**The lab**	One VM, a complete detection pipeline	2 min
3	**Workshop walkthrough**	Baseline, break, troubleshoot, prove, repair	9 min
4	**Best practices**	Health checks and operating habits to take home	5 min
5	**Get the lab, and questions**	Where to download it, then your questions	2 + 5 min

Most detection failures are operational

In real environments, detections go missing far more often because something broke than because a signature was never written.



Misconfiguration

A setting that is wrong, stale, or no longer matches the host



Broken pipelines

A link between sensor and SIEM that stopped passing data



Resource starvation

CPU, memory or disk that the sensor no longer has



Log ingestion issues

Events produced but rejected, dropped or misfiled

Training usually covers: rule writing and traffic analysis. It rarely covers: **recovery.**

"Running" is not a health check

What "running" tells you

- The process exists
- The container did not exit
- The dashboard tile is green

That is all it tells you.

What it can hide

- Zero rules loaded
- Listening on an interface with no traffic
- Unable to write its own logs
- Every event rejected by the SIEM
- A detection that has not fired in days

Health is a property of the whole chain, not of one process.

Six links, and any one of them can break



Each link depends on the one before it. One early failure makes everything after it fail too, so the order you check them in matters.

Every failure looks the same: silence



No error

Nothing pages anyone. The sensor did not crash.

No alert

The detection simply stops producing results.

No signal

A quiet dashboard looks the same as a safe network.

The question that matters: how long until someone notices?

The lab: one VM, a complete pipeline

Suricata 8.0.8

Captures on the lab interface with two custom rules

Lab traffic

A small capture replayed every five minutes

Fluent Bit

Ships eve.json to Splunk over HEC

Splunk Enterprise

Indexes events, mapped to CIM fields

Splunk detection

Scheduled alert on the lab rules

Student Lab Guide

Served on the VM at http://localhost/

On the desktop: **Break the Lab** · **Reset the Lab** · **Lab Guide** No internet access needed.

Learn what healthy looks like

STEP 1 OF 5

You cannot recognise broken until you have seen healthy. Students do this before touching anything.

```
$ /opt/surilab/scripts/validate-
health.sh
...
Results: 9 passed, 0 failed
[+] Lab is HEALTHY
```

Then walk the data path by hand: ten checks, one per link, noting what good output looks like.

The script says that something failed. The manual checks teach where to look.

1. Containers are up
2. Suricata started cleanly
3. Rules loaded: 2
4. Capture interface exists and is up
5. eve.json is fresh
6. Alerts are in the log
7. Shipper has no errors
8. HEC answers healthy
9. Events reach Splunk
10. The detection is firing

Break it

STEP 2 OF 5



Double-click Break the Lab

The lab is reset to healthy, then one thing is quietly broken. The only feedback is a notification: "Something just broke."

One fault per round

Five faults in all. Each comes up once before any repeats.

No clues given

No scenario number, no error popup. You start from symptoms.

Symptoms multiply

Three failed checks is usually one fault. Find the first broken link.

Faults cover configuration, rule loading, capture interface, permissions and the log pipeline.

Troubleshoot in data-path order

STEP 3 OF 5

Always in this order. Stop at the first "No": that is where the fault lives.

1	**Does Suricata start and stay running?**	**No** Configuration or syntax. Read the startup log.
2	**Do the rules load?**	**No** Ruleset missing, unreadable, or a rule is broken.
3	**Are packets being captured?**	**No** Interface wrong, down, or receiving nothing.
4	**Are logs being written?**	**No** Permissions, ownership or disk on the log directory.
5	**Are alerts being created?**	**No** A rule or traffic problem. Generate traffic, look again.
6	**Are logs reaching Splunk?**	**No** The shipper, the HEC token, or the index.
7	**Is the detection firing?**	**No** The alert's search, schedule or time window.

Prove it with evidence

STEP 4 OF 5

A hunch is not a root cause. For each round, students record a command and the output that proves it.

The worksheet

1. Observed symptom
2. Initial hypothesis
3. Evidence collected
4. Root cause, in one sentence
5. Fix applied
6. Validation

Example of evidence: ask the collector directly

```
$ curl -s localhost:8088/services/collector/event \
    -H 'Authorization: Splunk <token>' \
    -d '{"event":"test","index":"suricata"}'
{"text":"Incorrect index","code":7}
```

HTTP	HEC code	Meaning
200	0 Success	Event accepted
403	4 Invalid token	Token not known to Splunk
400	7 Incorrect index	Index missing or not allowed

Repair, then validate

STEP 5 OF 5

A fix is not finished until the whole data path is proven healthy again.



Health check

0 failed

Restart what you fixed, replay traffic, run the check



Fresh alerts

in Splunk

New alert events that arrived after your fix



Detection

firing again

The alert shows under Triggered Alerts

Done means all three. If any one is missing, you are not finished. Then click Break the Lab for the next round.

What building the lab taught me

The lab broke for real while I was building it. None of these produced an obvious error.

What I saw	What was wrong	The lesson
Sensor up, no line saying rules had loaded	The message is informational and hidden at the default log level	Know what your tools do not print by default
Container restarting in a loop, nothing useful logged	A read-only mount blocked a step the image runs at startup	Startup failures happen before logging starts
Alerts on disk, nothing in Splunk	The index did not exist, so every batch was rejected	A rejected event is not an error on the sending side
Shipper refused to start	A blank line in its config contained spaces	Whitespace is configuration

Health checks worth automating

Every fault in the lab is silent. Each of these turns a silent failure into an alert.

Alert when	It catches
Rules loaded is zero, or fewer than expected	A missing or broken ruleset after an update
Captured packet count stops increasing	A wrong, down or renamed capture interface
eve.json has not been updated in 60 seconds	Permission, disk and engine failures
No sensor events in the SIEM for 5 minutes	Any break in the shipping pipeline
The Suricata process or container restarts	Configuration errors and crash loops
A detection that normally fires goes quiet	All of the above, from the analyst's side

Operating habits that prevent the outage

Test before you restart

Run suricata -T against the config and rules before every reload

Pin your versions

Name exact releases. "latest" changes underneath you

Keep a known-good copy

Back up working configs, taken after a clean health check

Record the baseline

Write down what healthy output looks like, per check

Check in data-path order

Start at the packet and stop at the first broken link

Normalise your fields

Map alerts to a common model so detections survive tool changes

Get the lab

github.com/tskinnerarlo/When-Suricata-Breaks

In the repository

- README with quick start
- Student Lab Guide (Word)
- Link to the VM download

The VM

- OVA of about 18 GB, hosted on Google Drive
- SHA-256 hash in the README to verify it
- Download it before you need it

To run it

- VMware Workstation 17
- 8 GB memory and 4 cores for the VM
- About 45 GB of free disk

Four things to remember

1	**Running is not healthy**	Validate every link in the chain, not just the process.
2	**Silence is the symptom**	Detection failures do not announce themselves. Measure for quiet.
3	**Follow the data path**	Check in order and stop at the first broken link.
4	**Prove it, then validate it**	Evidence before the fix. End-to-end proof after it.



FIND IT · PROVE IT · FIX IT

Questions?

github.com/tskinnerarlo/When-Suricata-Breaks

Ted Skinner

Security Architect, Cisco